DDMO: Dynamic Detection of Malicious Model in the AI Supply Chain

Abstract—Malicious code poisoning in the AI model supply chain has emerged as a critical threat. Attackers embed hidden payloads into model artifacts distributed through open-source hubs, enabling arbitrary code execution upon deserialization or inference. Existing defenses rely on static signature scanning and predefined attack patterns, making them inherently vulnerable to evasion and zero-day exploits. We observe that benign AI models exhibit highly regular runtime execution templates, while malicious models inevitably deviate from these patterns. Building on this observation, we propose DDMO, a dynamic defense that does not require predefined attack patterns. DDMO combines kernel-level system call monitoring with Python-level semantic context and bridges the semantic gap between low-level runtime evidence and high-level model intent through cross-layer analysis. We evaluate DDMO on existing MalHug dataset and Advanced TensorAbuse Synthetic dataset spanning four attack categories. DDMO achieves an F1-score of 0.9870 on the MalHug and 1.0000 on the Advanced TensorAbuse Synthetic Dataset, outperforming all baseline methods. DDMO introduces only 5.69% average runtime overhead with 9.79 seconds detection latency, representing an acceptable cost for pre-deployment model vetting.

I. Introduction

Malicious code poisoning in the AI model supply chain has emerged as a critical and rapidly growing threat. Unlike traditional software, model files are opaque binary artifacts routinely downloaded and executed with minimal inspection from platforms such as Hugging Face [1]. Recent attack reports reveal that compromised models have already caused large-scale theft of user credentials and illicit exploitation of LLM API tokens, affecting hundreds of thousands of users and exposing CI/CD secrets of major service providers [2]–[4]. These incidents demonstrate that a single poisoned model can propagate broadly through the reuse-driven AI ecosystem, amplifying the blast radius of each attack [5].

To defend against these threats, existing research has focused on static detection methods that identify risks by scanning serialized files for known malicious signatures, decompiling pickle files, or detecting suspicious imports [6]–[8]. These tools reason about code structure rather than executed behavior, frequently confusing benign model operations with malicious attacks. For instance, if a benign model fetches remote weights and a malicious payload creates a reverse shell using the same networking library, such tools cannot distinguish the two [9]. Therefore there is a growing consensus that dynamic detection is necessary [10], [11]. However, existing dynamic methods rely on the knowledge of attack patterns to instrument specific functions or filter system events, making them vulnerable to rapidly evolving adversarial techniques.

The common shortcoming of these existing methods lies in their excessive dependence on predefined attack patterns. They have to rely on manually summarized attack features and fixed API filtering rules to identify threats. Static tools scan for known signatures or decompile bytecodes; dynamic methods hard-code which functions to hook based on expert heuristics and depend on static whitelists [12]. Given the rapid iteration of adversarial attack techniques, any defense contingent on knowing what to look for is inherently vulnerable, suffering from high false-positive and false-negative rates and complete defenselessness against zero-day exploits [13], [14].

To overcome this reliance on predefined attack patterns, we shift our strategy from hunting known threats to modeling benign behavioral characteristics. Our key observation is that benign AI models exhibit highly regular runtime templates, governed by the framework's internal logic and largely invariant across architectures and tasks. Normal execution consistently passes through a small set of stable phases such as deserialization, tensor materialization, and library loading. Malicious models inevitably deviate from these templates, producing anomalous patterns such as unexpected file access during deserialization or unauthorized network communication during inference.

Building on this observation, we propose DDMO, a defense that does not require predefined attack patterns. DDMO transforms dynamic model security protection into a two-stage behavior analysis problem, bridging the semantic gap between low-level system evidence and model intent. In the first stage, an unsupervised model learns the distribution of benign operation patterns from system call traces and filters out normal activity, requiring no attack signatures or whitelists. Only statistically abnormal operations are escalated to the second stage, where they are associated with Python semantic context collected via lightweight, non-invasive instrumentation. This cross-layer association combines kernel-level system call evidence with application-layer semantics. Finally, an LLM traces the root cause of underlying operations and judges whether they conform to the regular execution logic of the model or constitute malicious attacks. Through this two-stage pipeline, DDMO achieves both high detection accuracy and low false-positive rates while maintaining practical efficiency.

However, realizing DDMO faces three non-trivial technical challenges. First, how to define a meaningful unit of system call behavior that can be learned by a machine learning model. A single system call is excessively fine-grained and massive in volume, while malicious behaviors generally manifest as consecutive correlated call sequences [15], [16]. DDMO addresses this by aggregating system calls into lifecycle-bounded operations, converting raw traces into semantically meaningful operations. Second, how to achieve comprehensive semantic instrumentation across heterogeneous, rapidly evolving AI frameworks without source-code access or prohibitive overhead. DDMO leverages an LLM to dynamically generate framework-adaptive monkey-patching scripts that automatically discover and wrap public callable functions at runtime. Third, how to reliably align Python-level semantic events with kernel-level system call traces under high concurrency, where naive timestamp matching produces spurious associations. DDMO addresses this through a fuzzy spatio-temporal alignment strategy that combines temporal windowing with resource-related spatial hints to robustly associate each abnormal operation with its true application-layer context.

Experiments on 89 models from the MalHug dataset and 260 models from the advanced synthetic dataset demonstrate that DDMO achieves 100% precision, 98.70% F1-score, and 0% false positives on benign models from the advanced synthetic dataset, while introducing only 5.69% average runtime overhead. In summary, our contributions are:

- We propose the first hybrid dynamic detection system for AI model supply chain attacks that combines non-bypassable system-call evidence with Python semantic context, bridging the semantic gap between low-level behaviors and model intent.
- We develop a robust cross-layer association mechanism that aligns abnormal system-level operations with high-level semantic events, enabling LLM-based reasoning to provide accurate and explainable security alerts.
- Extensive experiments on real-world and synthetic malicious models demonstrate that our system achieves high detection accuracy and low false-positive rates, outperforming existing static defenses with good practical deployment value.
- We open source our system and provide a public benchmark of malicious models at https://anonymous.4open.science/r/DDMO-F184/ to facilitate future research in AI supply chain security.

II. Background and Motivation

A. AI Model Supply Chain Attack

The growth of open-source model hubs has transformed AI model development, enabling the community to download, fine-tune, and integrate pre-trained models into downstream applications [17]–[19]. This sharing paradigm introduces severe security vulnerabilities [20]. Recent reports have uncovered structural vulnerabilities on platforms such as Hugging Face [21], [22], and real-world attacks have surged [23]. As illustrated in Figure 1, the typical attack workflow involves uploading compromised models to public hubs that execute hidden payloads when downloaded. These attacks have already caused large-scale theft of user credentials and illicit exploitation of LLM API tokens, resulting in significant financial and operational damage [2].

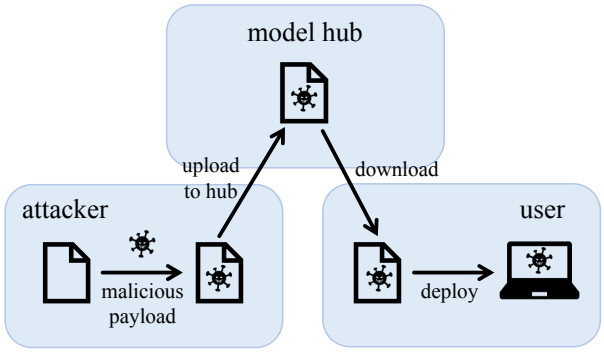


Fig. 1: AI model supply chain attack workflow.

AI model supply chains are vulnerable because malicious code is embedded within opaque binary artifacts [24], [25]. Attackers embed payloads directly into model structures using obfuscation techniques [26], disguising malicious code as normal model components or hiding it within unreadable binary formats to evade static security scanners [27].

Recent studies analyze malware poisoning in pre-trained model hubs [28], with particular attention to insecure serialization formats [29] that embed arbitrary execution commands triggered upon model loading. Since payload execution is integrated into the normal deserialization process, it bypasses conventional detection methods [30].

B. Existing Attack Vectors

Current attack methodologies targeting the AI model supply chain fall into two categories: insecure serialization attacks that embed executable payloads in model file formats, and framework-native attacks that weaponize legitimate framework APIs under the guise of normal tensor computations.

Insecure serialization attacks exploit the flexibility of model storage formats. The vulnerability level depends on the storage paradigm: formats preserving both architecture and weights pose the highest risk. Python's pickle and its derivatives, widely adopted by PyTorch and Scikit-learn, support arbitrary object instantiation and are highly vulnerable [31], [32]. TensorFlow's SavedModel and Keras's HDF5 [33] encapsulate execution graphs or custom layer definitions, presenting attack surfaces when loading untrusted artifacts. Formats restricted to raw numerical weights are inherently secure: Safetensors [34] strictly

TABLE I: Model formats and vulnerability to code injection.

Model Format	Primary Frameworks	Vulnerability Level
pickle, marshal	PyTorch, Scikit-learn	Highly Vulnerable
joblib, dill	Scikit-learn, Ray, PySpark	Highly Vulnerable
SavedModel	TensorFlow	Potentially Vulnerable
HDF5	Keras	Potentially Vulnerable
Checkpoint	TensorFlow	Secure
Safetensors	Hugging Face	Secure
ONNX	ONNX	Secure
JSON, MsgPack	Various	Secure

parses flat tensor data, and ONNX [35] relies on a rigid schema. Table I summarizes the security posture of widely used serialization formats.

Framework-native attacks weaponize legitimate framework APIs to execute malicious operations without obvious malicious syntax [36]. Malicious logic is compiled directly into the computational graph [37] rather than embedded as standalone scripts. This technique is exceptionally stealthy: Security monitors without application-layer context cannot distinguish legitimate from malicious operations.

C. Existing Defenses and Their Limitations

While extensive research addresses inference-time threats such as adversarial examples [38], [39] and data poisoning [40], [41], efforts to secure the AI supply chain against malware injection remain limited [9], [42]. Current defenses primarily rely on static analysis and signature-based scanning.

Fickling [7] decompiles pickle bytecodes into readable Python code to identify embedded malicious payloads. Hugging Face employs Picklescanning [8], which combines ClamAV [43] with import table inspection within pickle files. A static detection methodology for suspicious API calls in computational graphs was proposed by TensorAbuse [36].

Such static detection tools are inherently reactive and fail to fully capture the continuously evolving spectrum of attack vectors [44]. They address specific, previously observed vulnerabilities rather than adopting a holistic approach, severely restricting their ability to defend against zero-day exploits or novel attacks [45].

D. Motivation

To concretely illustrate why static, signature-based defenses fail against framework-native attacks, we present a real attack scenario derived from the TensorAbuse methodology [36]. Algorithm 1 shows the malicious logic embedded in a compromised model wrapper. The attack code is woven directly into the model's tf.Module initialization method and compiled into the computational graph alongside legitimate operations: the benign model setup (lines 1–4) constructs and loads the YAMNet audio classifier exactly as a legitimate model would, while the malicious payload (lines 5–10) leverages legitimate APIs, making it impossible for static analysis tools to distinguish it from benign code.

Algorithm 1: Framework-Native File Exfiltration Attack

```
1  Procedure YAMNet.__init__(W, P):
2      self.model ← yamnet_frames_model(P);
3      self.model.load_weights(W);
4      // — benign setup ends —
5      // — malicious payload begins —
6      path ← tf.io.matching_files("/home/*")[0];
7      content ← tf.raw_ops.ReadFile(filename =
         path);
8      prefix ← tf.constant("its a debug file\n");
9      payload ← tf.strings.join([prefix, content]);
10     tf.raw_ops.DebugIdentityV3(input =
         payload, debug_urls = grpc : // <
         malicious_url > );
```

The attack proceeds in three stages, each abusing a legitimate TensorFlow API so that no malicious syntax appears:

1) File discovery and reading. tf.io.matching_files expands a wildcard path pattern and returns the first match; tf.raw_ops.ReadFile then reads the entire file content into a tensor. Both APIs are officially provided by TensorFlow for dataset loading and routinely used in benign models, so their presence does not constitute a static signature of malice.
2) Content staging. tf.strings.join prepends a benign-looking prefix to the exfiltrated content. Since string manipulation is a standard tensor operation, this step is indistinguishable from legitimate logging or formatting at the API level.
3) Data exfiltration. tf.raw_ops.DebugIdentityV3, an official debugging primitive, sends the staged tensor to a remote endpoint via the grpc protocol. The operation is camouflaged as a debugging call and executes entirely within TensorFlow's native runtime, making the exfiltration channel invisible to Python-level import scanners.

Crucially, the entire payload is serialized into the SavedModel graph. Once the victim invokes tf.saved_model.load, the malicious operations execute as part of normal graph instantiation. Existing defenses fail to distinguish this attack: static scanners see only a computational graph composed of official TensorFlow ops and report no suspicious imports, while TensorAbuse's detector flags every use of ReadFile or DebugIdentityV3, misclassifying benign models that legitimately use these APIs for configuration loading or debugging. Neither approach reasons about what the operations actually do at runtime relative to the model's stated purpose.

III. Threat Model

In this paper, we focus on malicious code poisoning attacks within the AI model supply chain, in which attackers exploit the collaborative sharing characteristics of pre-trained model hubs to distribute tampered models embedded with malicious logic, thereby compromising victim host systems.

Attackers. The attackers are malicious AI model providers or actors who have compromised legitimate developer accounts [46]. They can create and upload malicious datasets and binary model files to public model hubs without providing the original training source code. To embed payloads, attackers exploit insecure serialization formats [32] or abuse hidden capabilities of framework APIs [36] to bypass static security scanners. They may also exploit leaked authentication tokens or hijack trusted identities to deceive the community [47]. Their primary goal is to execute unauthorized actions, such as establishing reverse shells, exfiltrating sensitive data, or deploying ransomware, without attracting attention.

Defenders: The defenders deploy DDMO to protect their systems against malicious models. DDMO operates in a controlled sandbox environment and is designed for two deployment scenarios. In the primary scenario, DDMO is integrated into a pre-deployment model vetting pipeline: before a model is released or integrated into any production system, it undergoes security vetting where DDMO monitors the model's behavior during loading and inference to detect embedded malicious payloads. To maximize coverage, the model should be exercised through all anticipated usage scenarios—including loading with expected input shapes and data types, inference across supported batch sizes, and edge-case inputs that stress runtime boundaries—ensuring that conditionally triggered malicious logic is activated and exposed during testing [48], [49]. In the secondary scenario, for environments where inference latency is not a critical constraint, DDMO can be deployed directly at model serving time to continuously monitor model behavior and block suspicious execution in real time.

IV. Approach

In this section, we present our approach for detecting malicious behaviors embedded in model artifacts. The pipeline operates in three stages: (i) Data Collection, which gathers two synchronized evidence streams—low-level system-call traces and high-level Python semantic logs; (ii) Coarse-grained Filtering, which clusters raw system calls into operation vectors, screens for anomalies via an unsupervised autoencoder, and retrieves aligned semantic context through fuzzy spatio-temporal matching; and (iii) Fine-grained Verification, where an LLM jointly reasons about the system-call evidence and Python semantic context to produce an intent score, a threat category, and an auditable evidence chain.

TABLE II: Security-relevant system-call categories used by our filter.

Param	Category	Purpose
file	File operations	Detect sensitive data leakage or configuration file tampering
network	Network operations	Detect data exfiltration or remote command execution
cred	Credential operations	Detect privilege-escalation behavior
desc	Descriptor operations	Detect file-related malicious behavior detection
signal	Signal handling	Capture malicious process termination attempts
clock	Time-related operations	Assist temporal anomaly analysis (e.g., abnormal delays)

A. Data Collection

The data collection stage simultaneously captures two synchronized streams of evidence during controlled model execution: a kernel-visible system-call stream and an application-layer semantic log stream. Together, these streams provide the complementary low-level and high-level signals needed for downstream analysis.

1) System-call collection.: We base our detection on system-call traces because they operate below the user-space level and cannot be bypassed by any Python-level obfuscation [50]: no matter how a model artifact hides its malicious intent through code mangling or dynamic loading, its actual interactions with the operating system must traverse the kernel and therefore leave a non-bypassable forensic record.

We employ a selective tracing strategy that focuses exclusively on security-relevant system-call families, discarding high-frequency but semantically uninformative calls to keep the trace volume manageable. During controlled model execution, we use strace to continuously trace system calls of the target process. To reduce noise and overhead, we immediately discard calls such as scheduler hints and memory-mapping operations that carry no security signal [51]. Table II summarizes the security-relevant syscall families we monitor and their roles in our detection pipeline. For example, file operations can reveal attempts to access sensitive data or modify configurations, while network operations may signal data exfiltration or remote command execution.

2) Instrumentation log collection.: To bridge the semantic gap between model-level intent and kernel-level behavior, we collect high-dimensional Python semantic context via non-invasive instrumentation. The key challenge is that heterogeneous ML frameworks expose frequently changing APIs, making manual per-framework instrumentation labor-intensive and coverage-incomplete. We address this through an LLM-assisted approach with two distinct phases.

Pre-execution: API discovery and script generation. Before model execution, we leverage an LLM to synthesize dynamic instrumentation scripts. A rigorously structured prompt constrains the LLM to generate code that inspects

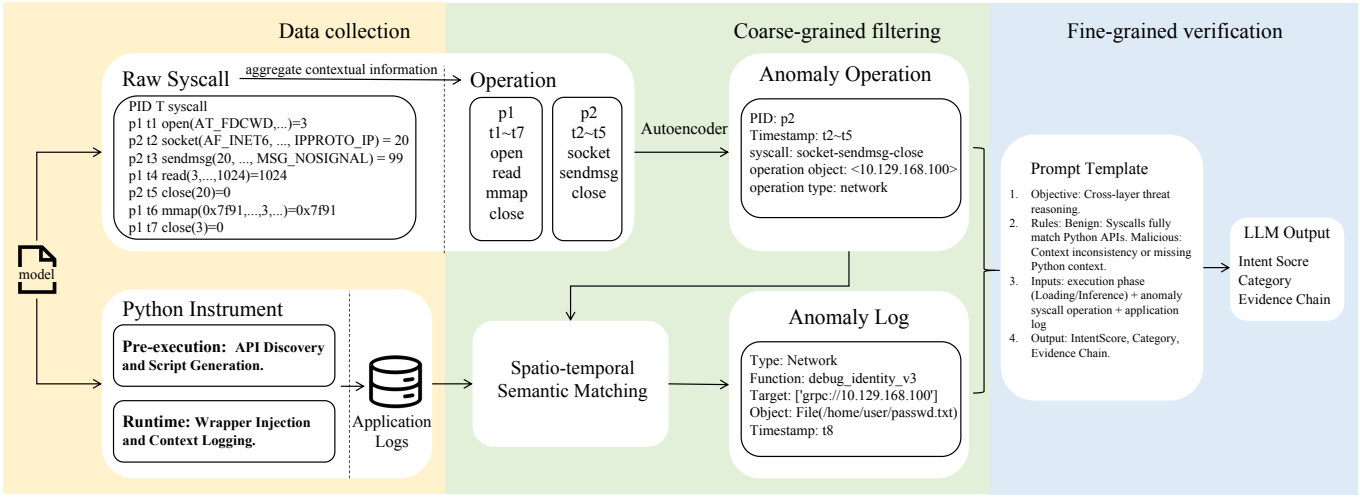


Fig. 2: Overview of the hybrid defense system.

the target module's namespace at startup, systematically identifies all public callable attributes, and constructs wrapper closures around each. The prompt specifies the security objective, mandates capturing execution context which contains timestamps, operation types, argument values and return types, and requires all intercepted events to be serialized into standardized JSON. This automated discovery eliminates statically hardcoded API lists and adapts to framework version changes by simply regenerating wrappers for the new API surface, without modifying the core detection pipeline. Our instrumentation intercepts all public callables of the target library, wraps only callable functions, and serializes arguments defensively, so it is neither a whitelist nor a blacklist. The prompt fixes a deterministic skeleton that works across frameworks, and the LLM rebuilds framework-specific security semantics on demand, which is what lets coverage track framework API evolution without manual per-framework coding.

Algorithm 2 formalizes this discovery and wrapping logic. The ApplyDynamicPatches procedure isolates public callables while skipping private attributes and non-callable objects, then generates wrapper closures for each target API.

Runtime: wrapper injection and context logging. At process startup, the generated script employs monkey patching (setattr) to replace original functions with wrapper closures. Thereafter, any invocation of a wrapped API triggers the injected closure, which captures the execution context and serializes it as a structured semantic event:

$$\ell_i = \langle \text{FuncName},\ \text{ArgsSummary} \rangle. \tag{1}$$

Here FuncName denotes the invoked framework or standard-library API, while ArgsSummary aggregates only lightweight information—tensor shapes and dtypes, configuration flags, truncated or hashed file paths, and bounded-length string summaries—controlling logging overhead while preserving rich contextual signals.

To ensure runtime safety, wrappers follow a conservative policy: they only wrap callable functions avoiding class definitions, built-in types, and C/C++ bindings, perform best-effort argument summarization without altering return values or side effects, and employ defensive serialization. The patching process is additionally wrapped in try-except blocks to survive read-only properties, immutable C-extension bindings, and dynamically generated descrip-

Algorithm 2: Automated Dynamic Instrumentation Logic

Input: Target library module $\mathcal{M}$ (e.g., torch, tensorflow)
Output: Instrumented module capable of context logging

```
Procedure ApplyDynamicPatches(M):
    foreach attribute name attr_name in dir(M) do
        if attr_name starts with "_" then
            continue
        target_func ← getattr(M, attr_name);
        if not is_callable(target_func) then
            continue
        wrapper ← Wrapper(target_func, attr_name);
        try;
            setattr(M, attr_name, wrapper);
        catch ReadOnlyAttributeError ;
            continue ;
Procedure Wrapper(func, name):
    return a closure that captures the full traceback, logs to JSON, executes func, and delegates return value;
```

tors that would otherwise crash the process. Any patch failure degrades gracefully without disrupting the target process.

B. Coarse-grained Filtering

After collecting raw evidence streams, the coarse-grained filtering stage transforms system-call traces into compact operation vectors, screens for anomalies, and retrieves the corresponding semantic context. This stage eliminates clearly benign activity before the expensive LLM reasoning stage.

1) System-call Clustering and Embedding: The collected system-call traces are extremely fine-grained and voluminous. A single model loading process may produce thousands of system calls, and meaningful malicious behavior invariably manifests as a sequence of related calls rather than as isolated events. To address this, we aggregate correlated system calls into compact high-level operation vectors to reduce trace noise and compress data volume, while providing standardized analysis units with complete semantics for anomaly detection.

To accurately model the execution semantics of a program, it is crucial to recognize that system calls inherently possess varying degrees of contextual dependency. Applying a uniform extraction strategy to all system calls would be ineffective: some operations are semantically self-contained and immediately indicative of high-risk behavior, whereas others are granular, stateful operations that remain ambiguous unless analyzed as a continuous sequence. Motivated by this fundamental difference, we propose a hybrid embedding scheme that categorizes the traced system calls into two distinct streams for targeted handling: (1) Directly malicious calls and (2) Stateful calls. By processing these two streams individually, we ultimately synthesize them into a unified feature space for downstream analysis.

Direct call embedding. The first category includes standalone, high-risk operations that do not require lifecycle context. We directly encode each of these calls into a fixed-dimension operation vector $\mathbf{v}_{\text{dir}} \in \mathbb{R}^d$. Specifically, we utilize a pre-trained BERT-based model to extract the rich semantic meaning from the call's underlying logic and literal arguments. This high-dimensional representation is subsequently projected down to the target dimension d, ensuring that semantically critical, self-contained threats are instantly captured without the need for sequence aggregation. The embedding is a BERT continued from bert-base-uncased, with hidden size 128, intermediate size 512, and 2 attention heads. We further pretrain it on the benign logs with masked language modeling and next-sentence prediction, and use its 128-dimensional [CLS] vector as the call embedding.

Stateful aggregation. Let $S = \{(s_k, \tau_k)\}_{k=1}^{N}$ denote the traced system-call stream, where s_k is the k-th syscall and τ_k its timestamp. For stateful operations, we maintain per-resource state keyed by $r = \langle \text{PID}, \text{FD} \rangle$, where PID denotes process ID, FD denotes a file or socket descriptor. For each key r, we collect the indices of calls that operate on this resource within one lifecycle segment,

$$I_r = \{\, k \mid (s_k, \tau_k) \in S,\ s_k \text{ uses resource } r,\ \tau^r_{\text{start}} \le \tau_k \le \tau^r_{\text{end}} \,\}, \tag{2}$$

where τ^r_{start} and τ^r_{end} are the timestamps of the resource-acquisition and resource-release calls (e.g., open / close or socket / shutdown). The lifecycle segment for resource r is then the ordered subsequence $S_r = \{(s_k, \tau_k)\}_{k \in I_r}$.

We aggregate this segment into a fixed-length operation vector by applying a feature-extraction function ϕ over all calls in the segment:

$$\mathbf{v}_{\text{sys}}(r) = \phi(S_r) \in \mathbb{R}^d, \tag{3}$$

where ϕ operates by extracting the common resource identifier $r = \langle \text{PID}, \text{FD} \rangle$ shared across the segment S_r, and concatenating it with the sequence of distinct, call-specific parameters (e.g., flags, buffer sizes, or return values) derived from each individual system call $s_k \in S_r$. This concatenated feature sequence is subsequently passed through an embedding layer to project it into the continuous $\mathbb{R}^d$ space. Intuitively, each $\mathbf{v}_{\text{sys}}(r)$ captures a dense, contextualized representation of the exact data flow and state transitions associated with a specific file or connection during its lifetime, effectively smoothing out scheduling artifacts and interleaving from other threads.

The final feature matrix combines $\mathbf{v}_{\text{dir}}$ and $\mathbf{v}_{\text{sys}}$, enabling the system to capture both direct and aggregated syscall behaviors. This hybrid representation ensures that directly malicious actions are immediately flagged, while stateful operations are analyzed in context to detect more subtle threats.

2) Autoencoder-based Anomaly Detection: We adopt unsupervised anomaly detection because benign model loading and inference produce a stable, learnable distribution of system-call operations, whereas malicious behaviors are inherently rare and diverse, making supervised classification impractical without labeled attack samples that cover the full threat space. Among unsupervised methods, we choose an Autoencoder for its conceptual simplicity and proven effectiveness in reconstruction-based anomaly detection: it learns to faithfully reconstruct benign operation vectors during training and naturally flags any vector it cannot faithfully reconstruct as anomalous.

We train an Autoencoder on operation vectors collected from benign model. Training uses system-call traces from ten benign Hugging Face models, namely bert-base-uncased, MiniLM-L6-H384-uncased, bert-base-chinese, distilgpt2, t5-small, resnet-50, emotion-english-distilroberta-base, whisper-tiny, twitter-roberta-base-sentiment, and tapas-base-finetuned-wtq. Each model is loaded once and runs one typical inference under strace. The Autoencoder has hidden widths 128 and 64 with a bottleneck of 32, decoding back to 128, and is optimized with MSE loss. We set the threshold θ

at the 99.9th percentile of normal reconstruction error. Given an operation vector $\mathbf{v}$, the Autoencoder computes a reconstruction error:

$$E = \|\mathbf{v} - \mathrm{Dec}(\mathrm{Enc}(\mathbf{v}))\|_2^2, \tag{4}$$

where Enc and Dec denote the encoder and decoder networks, respectively. Operations with reconstruction error E below a threshold θ are treated as consistent with normal model behavior and are discarded. If $E > \theta$, we mark the corresponding operation as abnormal, retain its full underlying system-call segment S' and metadata $\langle \mathrm{PID}, \mathrm{TID}, t_{\mathrm{start}}, t_{\mathrm{end}} \rangle$, and promote it to the subsequent spatio-temporal matching stage.

By performing this unsupervised prefiltering at the operation level, the system keeps the expensive cross-layer association and LLM reasoning off the critical path for the vast majority of benign activity, while still surfacing rare, suspicious low-level behaviors that deviate from the learned distribution of normal AI model execution.

3) Spatio-temporal Semantic Matching: After the autoencoder identifies abnormal operations, we must determine whether each anomaly can be explained by benign Python-level behavior. This requires retrieving the semantic context from the instrumentation log stream that corresponds to each abnormal system-call operation. However, naive one-to-one timestamp matching between system-call logs and semantic logs is unreliable in high-concurrency workloads. We therefore adopt a fuzzy spatio-temporal association strategy.

For each abnormal operation, the autoencoder screening stage provides the underlying system-call segment S' and its metadata $\langle \mathrm{PID}, t_{\mathrm{start}}, t_{\mathrm{end}} \rangle$. Given this metadata and the semantic log stream $L = \{(\ell_i, t_i)\}$, we perform filtering following the spatial-temporal matching rules below.

Temporal windowing. We apply a slack temporal window around the abnormal operation. For each candidate abnormal operation, we define its active time span $[t_{\mathrm{start}}, t_{\mathrm{end}}]$ based on the first and last system call in S', and search semantic events whose timestamps t_i fall into an expanded window $[t_{\mathrm{start}} - \Delta t, t_{\mathrm{end}} + \Delta t]$. The slack parameter Δt tolerates minor scheduling jitter while still localizing semantic context to the period when the resource was actually in use.

Spatial hints. Temporal proximity alone is insufficient under heavy concurrency. We therefore further filter candidates using resource-related hints derived from both layers, such as whether the operation is file- or network-related, file paths or prefixes, remote destinations, and other descriptor metadata when available. Combining temporal and spatial constraints yields a fuzzy but robust alignment that significantly reduces spurious associations when many model components execute in parallel.

C. Fine-grained Verification

Operations that survive coarse-grained filtering are escalated to the fine-grained verification stage, which jointly reasons about the low-level system-call evidence and high-level semantic context to produce a final intent judgment.

Determining whether abnormal low-level operations reflect benign model behavior or malicious intent is inherently challenging: the same system-call pattern can arise from legitimate model loading or from an attacker's attempt to exfiltrate data, and without semantic context the two are indistinguishable at the kernel level. To resolve this ambiguity, we feed each abnormal operation together with its aligned semantic contexts into an LLM, which judges whether the behavior is consistent with benign model execution. We use deepseek-v4-pro for cross-layer reasoning with temperature=0 and zero-shot prompting.

Prompt construction. We translate the aligned evidence into a structured natural-language prompt that includes: (i) a concise summary of the abnormal operation including operation type, key statistics, and risk-relevant parameters, (ii) the aligned semantic call contexts including library/API, call stack summary, and argument summaries, and (iii) the execution phase categorized into the model loading stage and inference stage.

Our maliciousness judgment follows a principled cross-layer reasoning rule: a behavior is classified as benign only when its low-level system-call activity can be plausibly explained by the aligned Python-level semantic context. Concretely, if the LLM identifies that the observed file reads, network connections, or process creations are consistent with the framework APIs and execution phase reported in the semantic log, the operation is deemed benign and discarded. Conversely, when inconsistencies exist between the Python-layer semantic logs and the corresponding system-call activities, or when no matching Python semantic context can be retrieved at all, the behavior is marked as malicious and escalated. This joint design also lets DDMO resist code-level obfuscation. Such obfuscation can disguise the Python-level semantics that instrumentation records, but it cannot alter the kernel-level system calls the model actually performs. When the code is obfuscated, the instrumentation emits misleading logs or none at all, while the kernel still records the real syscalls, and this misalignment between the two layers is exactly what the joint analysis flags. The TensorAbuse attacks in our dataset rely on this kind of disguise, and DDMO detects them successfully.

Decision and explanation. The LLM outputs IntentScore, a ThreatCategory label, and a rationale. Importantly, we persist the subset of semantic events and system-call summaries used in the reasoning as the evidence chain Context for auditing. If high-risk system-call patterns appear without any plausible semantic explanation, the system raises a high-severity alert and can block execution in the controlled environment.

V. Evaluation

We evaluate the effectiveness of DDMO in detecting malicious behavior embedded in model artifacts. We organize the evaluation around the following research questions:

- RQ1: What is the performance of DDMO in detecting malicious models, and does it show significant advantages over the baselines?
- RQ2: What is the runtime overhead of DDMO, and is it practical to deploy in real-world model workflows?
- RQ3: How does each part of DDMO contribute to the detection performance?

A. Experiment Setup

Datasets. We evaluate DDMO on two datasets spanning known deserialization exploits and advanced framework-native attacks:

- MalHug Dataset. This dataset consists of 89 anomalous models flagged by MalHug from public model repositories [52]. Manual source code auditing and sandbox verification label 78 as malicious—mainly abusing Pickle deserialization to implant reverse shells—and 11 as benign proof-of-concept samples that verify exploitability without destructive operations. While these represent real-world threats, their methodologies rely on well-known deserialization flaws and are straightforward to detect with existing static scanners. Our detection target is whether a model actually executes harmful behavior. The eleven proof-of-concept models in MalHug only demonstrate that a vulnerability is exploitable and perform no destructive operation, so we label them benign rather than malicious under this definition.
- Advanced TensorAbuse Synthetic Dataset. To evaluate our system against highly stealthy, framework-native attacks, we constructed a second dataset using the TensorAbuse framework [36]. This corpus comprises two complementary subsets. The malicious subset contains 160 models: for each of 10 diverse foundation models, we embed 16 attack variants spanning four categories using TensorAbuse primitives. The benign subset contains 100 models that invoke the same high-risk TensorFlow APIs for legitimate purposes such as outputting debug tags, constructed to precisely quantify the false-positive rate. The autoencoder training set and the evaluation set are not identical. Their architectures partly overlap, since both include BERT-style encoder-only transformers, but the evaluation set also contains architectures absent from training such as YAMNet, VGG16, and EfficientNet, so evaluation is not limited to same-architecture samples.

It is worth emphasizing that the construction method of the advanced TensorAbuse synthetic dataset is as follows. TensorAbuse research categorizes native TensorFlow primitives into five functional capability groups: Arbitrary File Read, Arbitrary File Write, Directory Read, Network Send and Network Receive. Drawing on these categories, we design four distinct attack variants for evaluation: File Leakage, IP Exposure, Code Execution and Shell Access.Each attack type is assembled by chaining APIs from specific capability categories. Taking File Leakage as an example, its attack chain works as follows: Directory Read to locate the target file, Arbitrary File Read to load its content, and Network Send to exfiltrate data to a remote endpoint.

For each attack instance, we randomly sample one concrete API from each required category from the pool of officially supported TensorFlow primitives described in the TensorAbuse study. The selected APIs are then woven into the initialization or inference path of a benign foundation model so that malicious operations execute as part of normal graph construction or tensor flow. Because every API in the chain is an official TensorFlow operation, the resulting model artifact contains no suspicious imports, no non-standard syntax, and no standalone malicious scripts—only a computational graph composed entirely of framework-native operations. For the benign subset, we deliberately invoke the same high-risk APIs for their legitimate purposes, such as reading local dataset configurations or logging debug information, creating a rigorous benchmark where any detector that relies solely on the presence of high-risk APIs cannot distinguish malicious models from benign ones.

For all datasets, the model artifact is the unit under test. Labels are assigned by manual payload inspection.

Baselines. We compare against four baseline detection methods: three widely adopted security scanners integrated into the Hugging Face platform including JFrog [53], Protect AI [6], and ClamAV [43], along with the specialized detection tool from TensorAbuse [36]. For JFrog, Protect AI, and ClamAV, we uploaded our model artifacts to the Hugging Face platform, triggering its live security scanning pipelines to capture authentic detection efficacy in a real-world supply chain setting. For TensorAbuse, we use the authors' open-source implementation. We also examined a dynamic detection method [24], but did not include it as a baseline. Its detection relies on a blacklist of a few system calls such as socket, connect, execve, and chmod, and calls like execve appear in virtually every normal model load, which drives its false-positive rate on benign models to 100% and forces manual review of the flagged calls.

All experiments run on a workstation with dual Intel Xeon Silver 4210R CPUs, 128 GB RAM, and two NVIDIA GeForce RTX 3090 GPUs.

B. RQ1: Detection Performance

To answer this research question, we evaluate the detection performance of DDMO and the baseline methods across the two datasets.

TABLE III: Detection performance of DDMO and baselines across both datasets.

Method	MalHug			Advanced TensorAbuse Synthetic		
	Precision	Recall	F1	Precision	Recall	F1
JFrog	0.8953	0.9872	0.9390	0.8889	0.3750	0.5275
Protect AI	0.8966	1.0000	0.9455	0.6154	1.0000	0.7619
ClamAV	0.8904	0.8333	0.8609	–	0.0000	–
TensorAbuse	–	–	–	0.6154	1.0000	0.7619
DDMO	1.0000	0.9744	0.9870	1.0000	1.0000	1.0000

TABLE IV: Detected malicious models on the TensorAbuse Synthetic Dataset by attack type (40 per type).

Method	Attack Type (Total models)				Total Detected
	File Exfil.	IP Exposure	Code Insertion	Remote Shell	
JFrog	40 / 40	0 / 40	0 / 40	20 / 40	60 / 160
Protect AI	40 / 40	40 / 40	40 / 40	40 / 40	160 / 160
ClamAV	0 / 40	0 / 40	0 / 40	0 / 40	0 / 160
TensorAbuse	40 / 40	40 / 40	40 / 40	40 / 40	160 / 160
DDMO	40 / 40	40 / 40	40 / 40	40 / 40	160 / 160

Table III summarizes the detection performance across both datasets. On the MalHug Dataset, traditional static scanners achieve moderate to high F1 scores by matching known deserialization signatures. DDMO achieves a marginal improvement via accurate intent verification. However, on the Advanced TensorAbuse Synthetic Dataset, the performance gap expands dramatically. Static tools suffer a sharp drop in detection performance. While TensorAbuse achieves a perfect recall, its precision is only 0.6154, as it fails to distinguish legitimate and malicious calls to TensorFlow APIs. Since Protect AI integrates a scanning mechanism similar to TensorAbuse, it delivers highly comparable detection performance. Only DDMO maintains consistently high performance on both datasets, achieving an F1 of 0.9870 on the MalHug Dataset and a perfect 1.0000 on the Advanced TensorAbuse Synthetic Dataset.

Performance on the MalHug dataset. Traditional static detection tools such as JFrog, Protect AI, and ClamAV achieve relatively high recall rates by relying on established vulnerability signatures. However, these baselines exhibit a critical limitation: they fail to distinguish actual weaponized attacks from the 11 benign proof-of-concept models. By simply classifying any model containing a vulnerable signature as malicious, these methods generate substantial false positives.

In contrast, DDMO successfully detects 76 out of 78 genuinely malicious models while accurately categorizing all 11 proof-of-concept models as non-malicious. Because DDMO evaluates the actual intent of dynamic behavior, it demonstrates robust resilience against false positives. Notably, the TensorAbuse detector fails to identify any of these models, as its static graph-based approach cannot detect deserialization exploits triggered before computational graph initialization or outside the scope of the computational graph.

Performance on the advanced TensorAbuse synthetic dataset. Table IV provides a detailed breakdown of the detection results. Protect AI and the TensorAbuse detection tool successfully detect 100% of the malicious models. The TensorAbuse tool achieves perfect detection here because these attacks were generated using its own primitive methodologies, making them highly visible to its static API-checking mechanism. The same to Protect AI, which integrates a scanning mechanism similar to TensorAbuse.

DDMO achieves this 100% detection rate without relying on static signatures or pre-defined API allowlists. It constructs statistical behavioral baselines from benign model execution and dynamically intercepts anomalous system calls triggered by malicious payloads, enabling precise recognition of malicious intent irrespective of attack vectors. The performance of JFrog and ClamAV varies significantly across attack categories, highlighting the limitations of traditional scanning tools against sophisticated, framework-native manipulations. DDMO also handles multi-step attack chains. Each step in such a chain already deviates from benign behavior on its own, since a chmod, a sensitive file read, or a socket send is flagged by the autoencoder, and the Python log then supplies the context that links the steps at the semantic level, such as which API caused the exfiltration and which target IP it reached. The File Leakage attacks in our dataset are exactly such a read-then-exfiltrate chain, and DDMO detects them.

To further verify DDMO's capability against false positives, we also evaluate all methods on benign models. The precision results on the Advanced TensorAbuse Synthetic Dataset in Table III clearly reveal the drawbacks of static detection mechanisms. TensorAbuse and Protect AI misclassify all 100 benign models as malicious, resulting in a 100% false positive rate. It relies solely on static scanning against a predefined blacklist of high-risk APIs and lacks the ability to understand API semantics and invocation context. Similarly, JFrog also produces numerous misclassifications, indicating that signature-based detection methods fail when legitimate operations overlap with known attack patterns.

In contrast, DDMO successfully identifies all 100 models as benign. By leveraging runtime dynamic monitoring, DDMO recognizes that the system calls generated by these APIs align with the established statistical baseline of normal framework operations. When a suspicious operation triggers cross-layer analysis, the LLM-based reasoning engine interprets high-level Python semantics to validate benign intent. This demonstrates that DDMO bridges the semantic gap, providing accurate defense without impeding legitimate workflows.

Overall, experimental results on both datasets prove that DDMO is a robust defense solution with strong generalization capability. It accurately detects malicious AI models exploiting common deserialization vulnerabilities or sophisticated computational graph tampering. Furthermore, equipped with cross-layer semantic reasoning,

TABLE V: Runtime overhead and detection latency of DDMO.

Model	Execution Time (s)			Detection
	Vanilla	w/ MAD	Overhead	Latency (s)
BERT	11.4857	12.1767	6.02 %	13.5014
YamNet	16.3302	17.1434	4.98 %	10.3641
EfficientNet	24.4290	25.8889	5.97 %	6.5911
VGG16	25.0444	26.4766	5.46 %	8.7171
Average	19.3223	20.4214	5.69 %	9.7934

DDMO effectively differentiates legitimate usage of sensitive framework APIs from malicious attacks, achieving high precision on both datasets. This generalization stems from the fact that DDMO encodes no predefined attack patterns. The autoencoder is trained unsupervised on benign traces, and the LLM only checks whether a suspicious system call can be explained by its Python semantics, so the same detector transfers across attack types without modification. Detecting both the Pickle-based MalHug attacks and the framework-native TensorAbuse attacks with one model supports this claim.

C. RQ2: Efficiency and Deployability

To evaluate the practical deployability of DDMO, we investigate its impact on host system performance and responsiveness to threats. We do not compare DDMO with the baseline methods here, as the baselines are strictly static analysis tools designed to scan model artifacts pre-deployment, whereas DDMO is a dynamic defense mechanism. Comparing the one-time scanning duration of static tools against the continuous runtime overhead of a dynamic monitor is fundamentally inequitable. Therefore, our evaluation focuses on the absolute performance impact of DDMO on standard model execution.

We assess efficiency using two primary metrics:

- Performance Overhead: The degradation in model loading time and inference throughput (Samples/Second) caused by deploying DDMO.
- Detection Latency: The time elapsed from the exact moment a malicious operation is initiated by the model to the generation of the final alert.

Performance overhead analysis. The performance impact of DDMO on normal model execution is presented in Table V. DDMO introduces marginal overhead: the total execution time increases by an average of only 5.69%, with the overhead consistently remaining within a narrow 4.9% to 6.0% margin across fundamentally different model architectures.

This performance penalty is attributed to our architectural design. First, the application-layer dynamic instrumentation employs a lightweight boundary-hooking mechanism that only captures high-level metadata instead of deeply traversing and serializing complex, multi-gigabyte Tensor objects. More importantly, the Autoencoder screening module and the LLM reasoning module run entirely asynchronously on the CPU. Because they do not compete for GPU memory or compute cycles, the intensive matrix multiplications and data parallelism required for AI inference remain completely unhindered.

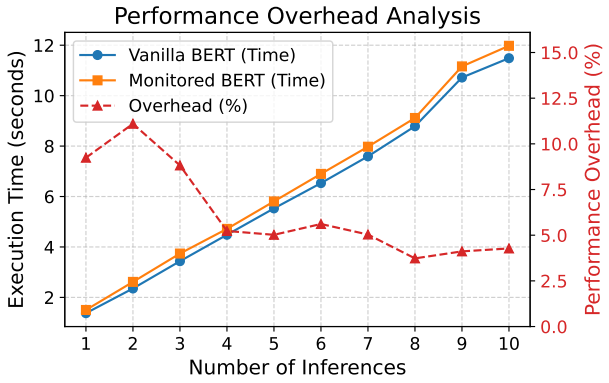


Fig. 3: Execution time of Vanilla vs. Monitored BERT and relative overhead across inferences.

To further demonstrate that our detection system minimally impacts inference efficiency, we analyzed the relationship between the number of inference iterations and relative latency overhead. Figure 3 illustrates the execution time of the BERT model over 1 to 10 inference steps, comparing vanilla execution with DDMO-monitored execution. The relative overhead percentage is highest during the initial executions but stabilizes at a lower baseline as the number of inferences increases.

This observation conforms to the inherent runtime behavior of deep learning frameworks. During model loading and initialization, the framework emits a dense stream of system calls, such as memory mapping operations and heavy file I/O for loading model weights, which introduces transient CPU-heavy monitoring overhead. In contrast, after the model transitions to steady inference, most compute-intensive workloads are offloaded to hardware accelerators. Such computation generates minimal new system calls and consumes negligible host CPU resources. Consequently, the performance overhead brought by dynamic instrumentation and real-time syscall tracing gradually subsides.

Detection latency analysis. As shown in Table V, DDMO achieves an average detection latency of 9.79 seconds. This latency encompasses the full analytical pipeline: capturing low-level system calls, completing the Autoencoder forward pass, extracting the corresponding Python contextual logs, and generating the final LLM intent classification. The LLM intent classification is only invoked on a small fraction of operations marked as abnormal by the Autoencoder, so its cost does not scale with total runtime but solely with the number of suspicious events.

Although dynamic cross-layer reasoning inevitably introduces slight latency compared with lightweight signature matching, this two-stage architecture constrains the

TABLE VI: Ablation study of DDMO.

Method	Precision	Recall	F1	Latency (s)	Token Cost
DDMO-S	90.91%	44.30%	59.57%	30.3126	867075
DDMO-L	85.25%	99.16%	91.68%	8.9778	50005
DDMO	100.00%	99.16%	99.58%	9.7934	57207.1

overall performance overhead within a practical range, making our approach fully applicable to offline model vetting and pre-deployment security auditing pipelines.

D. RQ3: Component Ablation Study

In this section, we evaluate the contribution of each core component of DDMO through ablation studies. To demonstrate the necessity of our two-stage cross-layer architecture, we evaluate two variants:

- DDMO–S: which removes the unsupervised Autoencoder screening stage, directly feeding all raw system call windows and application-level instrumentation logs to the LLM for anomaly detection;
- DDMO–L: which removes the application-level dynamic instrumentation, relying solely on the anomalous system call operations to make its final judgment;

We extensively evaluate these two ablated variants and compare their performance with the full DDMO model in terms of precision and runtime efficiency. Detailed quantitative results are presented in Table VI. Overall, both variants suffer from substantial performance degradation or prohibitive runtime overhead compared with the complete pipeline, demonstrating that both stages are indispensable to the overall framework.

Specifically, DDMO-S suffers from a catastrophic decrease in detection efficiency and a massive surge in LLM token consumption. Because it bypasses the Autoencoder screening, the LLM is overwhelmed by the massive volume of benign, repetitive system events. This increases the average detection latency and incurs extreme API costs. It also slightly reduces precision, as the LLM is more prone to hallucination when processing excessive background noise. This demonstrates that the statistical screening stage is crucial for filtering out normal activity and maintaining practical deployability.

On the other hand, DDMO-L performs the worst in precision, suffering a severe drop. By removing the application-level instrumentation logs, the LLM is forced to judge intent based entirely on low-level system calls. Without high-level semantic context such as function arguments to explain why a system call was executed, benign operations are frequently misclassified as malicious. This confirms that dynamic cross-layer context alignment is an important component for eliminating false positives and accurately discerning the true intent of AI models. We also checked whether the result depends on a specific LLM by swapping in three different models. On MalHug all of them reach an F1 of 0.9870, and on the TensorAbuse Synthetic Dataset all reach precision 1.0 with recall of 100%, 100%, and 98.75% respectively. The detection paradigm, not the LLM's capability, drives the performance, since the LLM only verifies whether an abnormal system call is explainable by its Python semantics.

VI. Related Work

Malicious code attacks in the AI supply chain. Malicious code attacks in the AI supply chain include insecure serialization attacks and framework-native attacks. Serialization mechanisms that support arbitrary object instantiation blur the boundary between data and executable code [54], [55], enabling payload execution upon loading compromised artifacts—a threat that remains widespread on public hubs [24], [56], [57]. More recently, framework-native attacks exploit lower-level APIs with file system and network access [58] to exfiltrate data or execute unauthorized commands without obvious malicious syntax, as demonstrated by TensorAbuse [36], which embeds attacks into the computational graph using officially supported operations.

Defenses against AI supply chain attacks. Current defenses rely heavily on static analysis and signature-based scanning [7], [8], but these approaches fail to capture dynamic execution semantics and cannot establish the semantic context needed to distinguish benign from malicious intent [59]. Zhu et al. [36] made progress toward semantic-aware detection, yet their method still operates at a static level and struggles with API usage ambiguity in practice [60].

System call analysis has been widely validated as a powerful paradigm for conventional malware detection [61], [62], yet has not been adapted to AI model supply chain threats. API2Vec++ [63] uses temporal and attribute graph embeddings to characterize contextual behaviors across processes. API-MalDetect [64] applies CNNs and BiGRUs to API sequences for deep semantic feature extraction. CruParamer [65] incorporates parameter sensitivity into API embeddings.

VII. Discussion

Limitation. DDMO is designed to detect malicious behaviors that a model artifact executes against the host system by observing runtime interactions between the model process and the operating system. It cannot address attacks that target the model itself without affecting the host system, such as neural backdoor injection, where malicious functionality is embedded into learned parameters. These attacks compromise prediction integrity rather than runtime behavior, producing only normal tensor computations that generate no anomalous system calls or suspicious API invocations. Defending against such threats requires complementary techniques including model provenance verification, weight integrity checking, and robust training, which are orthogonal to and compatible with DDMO's host-level defense. Dynamic detection also shares a general limitation. Conditionally triggered malicious logic runs

only when its trigger condition is met during testing, so logic that is never activated goes undetected. Our threat model therefore recommends exercising each model across all anticipated usage scenarios during testing, including varied input shapes and edge-case inputs, to activate conditionally triggered logic and maximize coverage. We also plan a larger benign evaluation on public model hubs to stress the false-positive rate further, since the current results are based on the benign set described above.

Relationship with static detection. Static scanning tools and DDMO serve complementary roles in supply chain defense. Static scanners analyze model artifacts without executing them, making them lightweight and well-suited for platform-level screening of uploaded models. However, because they lack runtime context, they cannot reason about API usage semantics and are fundamentally blind to framework-native attacks that use only legitimate operations. DDMO operates at a deeper level by executing the model in a sandboxed environment to observe actual runtime behavior, enabling it to detect threats that evade static analysis. The two approaches can be combined in a defense-in-depth strategy: static scanning serves as an efficient first-pass filter to eliminate known exploit patterns, while DDMO provides thorough pre-deployment vetting for models that pass the initial screen.

VIII. Conclusion

Malicious model artifacts present a growing threat to the AI supply chain: attackers embed harmful payloads within serialized models that execute during loading or inference, evading static scanners by using legitimate framework APIs. In this paper, we propose DDMO, a hybrid runtime defense system that combines non-bypassable kernel-level system-call monitoring with LLM-assisted Python semantic instrumentation to detect malicious behaviors in model artifacts. DDMO captures actual runtime execution evidence through a two-stage architecture: an unsupervised Autoencoder prefilter that surfaces anomalous low-level operations, and a cross-layer LLM reasoning engine that aligns these operations with their high-level semantic context to determine malicious intent. We evaluate DDMO on the MalHug Dataset and advanced synthetic attacks. The results show that DDMO achieves near-perfect detection with only 5.69% average runtime overhead, demonstrating that it provides a practical, deployable defense for the AI model supply chain.

References

[1] H. Face. (2026) Hugging face – the ai community building the future. https://huggingface.co/. Https://huggingface.co/.

[2] P. Girnus, F. Tucci, D. Patel, S. Dulude, A. Verma, and J. R. Navato. (2026) Your ai gateway was a backdoor: Inside the litellm supply chain compromise. https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html. Access:2026-05-18. [Online]. Available: https://www.trendmicro.com/en_us/research/26/c/inside-litellm-supply-chain-compromise.html

[3] P. P. Index. (2026) Incident report: Litellm/telnyx supply-chain attacks, with guidance. https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/. Access:2026-05-18. [Online]. Available: https://blog.pypi.org/posts/2026-04-02-incident-report-litellm-telnyx-supply-chain-attack/

[4] unit42. (2026) Weaponizing the protectors: Teampcp's multi-stage supply chain attack on security infrastructure. https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/. Access:2026-05-18. [Online]. Available: https://origin-unit42.paloaltonetworks.com/teampcp-supply-chain-attacks/

[5] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2087–2098. [Online]. Available: https://doi.org/10.1145/3691620.3695271

[6] Protect AI. (2021) Modelscan: Protection against model serialization attacks. Access:2026-05-18. [Online]. Available: https://github.com/protectai/modelscan

[7] Trail of Bits. (2021) fickling. Access:2026-05-18. [Online]. Available: https://github.com/trailofbits/fickling

[8] Hugging Face. (2024) Pickle scanning. Access:2026-05-18. [Online]. Available: https://huggingface.co/docs/hub/security-pickle#pickle-scanning

[9] M. Nabeel and O. Starov, "Deep dive into the abuse of dl apis to create malicious ai models and how to detect them," 2026. [Online]. Available: https://arxiv.org/abs/2601.04553

[10] N. Z. Gorment, A. Selamat, L. K. Cheng, and O. Krejcar, "Machine learning algorithm for malware detection: Taxonomy, current challenges, and future directions," IEEE Access, vol. 11, pp. 141 045–141 089, 2023.

[11] W. Guo, H. Wen, L. Kong, J. Xue, J. Hu, W. Han, and Y. Wang, "A survey on malware analysis with large language models," in Knowledge Science, Engineering and Management, T. Zhu, W. Zhou, and C. Zhu, Eds. Singapore: Springer Nature Singapore, 2026, pp. 41–52.

[12] A. Virkud, M. A. Inam, A. Riddle, J. Liu, G. Wang, and A. Bates, "How does endpoint detection use the MITRE ATT&CK framework?" in 33rd USENIX Security Symposium (USENIX Security 24). Philadelphia, PA: USENIX Association, Aug. 2024, pp. 3891–3908. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/virkud

[13] Y. Guo, "A review of machine learning-based zero-day attack detection: Challenges and future directions," Computer Communications, vol. 198, pp. 175–185, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0140366422004248

[14] H. Aghakhani, F. Gritti, F. Mecca, M. Lindorfer, S. Ortolani, D. Balzarotti, G. Vigna, and C. Kruegel, "When Malware is Packin' Heat; Limits of Machine Learning Classifiers Based on Static Analysis Features," in Network and Distributed System Security Symposium. San Diego, United States: Internet Society, Feb. 2020. [Online]. Available: https://hal.science/hal-04093553

[15] P. Guo, "Intrusion detection based on complete system call information," in Proceedings of the 2024 International Conference on Digital Society and Artificial Intelligence, ser. DSAI '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–5. [Online]. Available: https://doi.org/10.1145/3677892.3677893

[16] L. Arnoud, V. Breux, P.-H. Thevenon, and É. Gaussier, "Sok: An in-depth analysis of intrusion detection systems based on system calls," Journal of Cybersecurity and Privacy, vol. 6, no. 3, 2026. [Online]. Available: https://www.mdpi.com/2624-800X/6/3/99

[17] L. Gambacorta and V. Shreeti, "The ai supply chain," Review of Network Economics, vol. 24, no. 4, pp. 205–223, 2025. [Online]. Available: https://doi.org/10.1515/rne-2025-0063

[18] W. Jiang, N. Synovic, M. Hyatt, T. R. Schorlemmer, R. Sethi, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "An empirical

study of pre-trained model reuse in the hugging face deep learning model registry," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2463–2475.

[19] M. Taraghi, G. Dorcelus, A. Foundjem, F. Tambon, and F. Khomh, "Deep learning model reuse in the huggingface community: Challenges, benefit and trends," in 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2024, pp. 512–523.

[20] G. Digregorio, M. Di Gennaro, S. Zanero, S. Longari, and M. Carminati, "When secure isn't: Assessing the security of machine learning model sharing," 09 2025.

[21] Z. Ding, Q. Fu, J. Ding, G. Deng, Y. Liu, and Y. Li, "A rusty link in the ai supply chain: Detecting evil configurations in model repositories," 2025. [Online]. Available: https://arxiv.org/abs/2505.01067

[22] T. Stalnaker, N. Wintersgill, O. Chaparro, L. A. Heymann, M. D. Penta, D. M. German, and D. Poshyvanyk, "An empirical analysis of machine learning model and dataset documentation, supply chain, and licensing challenges on hugging face," 2025. [Online]. Available: https://arxiv.org/abs/2502.04484

[23] HiddenLayer. (2026) Ai threat landscape 2026. https://www.hiddenlayer.com/report-and-guide/threatreport2026. Access:2026-05-18. [Online]. Available: https://www.hiddenlayer.com/report-and-guide/threatreport2026

[24] B. Casey, J. C. S. Santos, and M. Mirakhorli, "A large-scale exploit instrumentation study of ai/ml supply chain attacks in hugging face models," 2024. [Online]. Available: https://arxiv.org/abs/2410.04490

[25] M. L. Siddiq, T. H. Romel, N. Sekerak, B. Casey, and J. C. S. Santos, "An empirical study on remote code execution in machine learning model hosting ecosystems," 2026. [Online]. Available: https://arxiv.org/abs/2601.14163

[26] Z. Wang, C. Liu, X. Cui, J. Yin, and X. Wang, "Evilmodel 2.0: Bringing neural network models into malware attacks," Computers & Security, vol. 120, p. 102807, 2022. [Online]. Available: http://dx.doi.org/10.1016/j.cose.2022.102807

[27] A. Kathikar, A. Nair, B. Lazarine, A. Sachdeva, and S. Samtani, "Assessing the vulnerabilities of the open-source artificial intelligence (ai) landscape: A large-scale analysis of the hugging face platform," in 2023 IEEE International Conference on Intelligence and Security Informatics (ISI), 2023, pp. 1–6.

[28] W. Jiang, N. Synovic, R. Sethi, A. Indarapu, M. Hyatt, T. R. Schorlemmer, G. K. Thiruvathukal, and J. C. Davis, "An empirical study of artifacts and security risks in the pre-trained model supply chain," in Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses, ser. SCORED'22. New York, NY, USA: Association for Computing Machinery, 2022, p. 105–114. [Online]. Available: https://doi.org/10.1145/3560835.3564547

[29] N.-J. Huang, C.-J. Huang, and S.-K. Huang, "Pain pickle: Bypassing python restricted unpickler for automatic exploit generation," in 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), 2022, pp. 1079–1090.

[30] T. A. Nguyen, T. H. M. Le, and M. A. Babar, "Securing the ai supply chain: What can we learn from developer-reported security issues and solutions of ai projects?" 2026. [Online]. Available: https://arxiv.org/abs/2512.23385

[31] Python Software Foundation. (2026) pickle — python object serialization. https://docs.python.org/3/library/pickle.html. Accessed: 2026-05-25.

[32] T. Liu, G. Meng, P. Zhou, Z. Deng, S. Yao, and K. Chen, "The art of hide and seek: Making pickle-based model supply chain poisoning stealthy again," 2025. [Online]. Available: https://arxiv.org/abs/2508.19774

[33] The HDF Group. High-performance data management and storage suite. https://www.hdfgroup.org/HDF5/. Accessed: 2026-05-25.

[34] safetensors. Safetensors. https://github.com/huggingface/safetensors. Accessed: 2026-05-25.

[35] onnx. (2020) ONNX. https://github.com/onnx/onnx. Accessed: 2026-05-25.

[36] R. Zhu, G. Chen, W. Shen, X. Xie, and R. Chang, " My Model is Malware to You: Transforming AI Models into Malware by Abusing TensorFlow APIs ," in 2025 IEEE Symposium on Security and Privacy (SP). Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 449–466. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00012

[37] H. Langford, I. Shumailov, Y. Zhao, R. Mullins, and N. Papernot, "Architectural neural backdoors from first principles," 2024. [Online]. Available: https://arxiv.org/abs/2402.06957

[38] H. Wu, G. Ou, W. Wu, and Z. Zheng, "Improving transferable targeted adversarial attacks with model self-enhancement," in 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 24 615–24 624.

[39] P. Xie, Y. Bie, J. Mao, Y. Song, Y. Wang, H. Chen, and K. Chen, "Chain of attack: On the robustness of vision-language models against transfer-based adversarial attacks," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2025, pp. 14 679–14 689.

[40] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, "Poisoning web-scale training datasets is practical," in 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 407–425.

[41] Z. Chen, Z. Zhao, S. K. Dutta, C. Lin, C. Shen, and X. Zhang, "Generalizable targeted data poisoning against varying physical objects," 2025. [Online]. Available: https://arxiv.org/abs/2412.03908

[42] X. Jiang, S. Yang, W. Yang, Y. Liu, and C. Ji, "Sok: A taxonomy of attack vectors and defense strategies for agentic supply chain runtime," 2026. [Online]. Available: https://arxiv.org/abs/2602.19555

[43] Cisco Systems. ClamAVNet. https://www.clamav.net/. Access:2026-05-25. [Online]. Available: https://www.clamav.net/

[44] Z. Qian, F. Zhong, Q. Hu, Y. Jiang, J. Huang, M. Ren, and J. Yu, "Software vulnerability analysis across programming language and program representation landscapes: A survey," 2025. [Online]. Available: https://arxiv.org/abs/2503.20244

[45] M. G. Tehrani, E. Sultanow, W. J. Buchanan, M. Houmani, and C. H. D. Fodja, "Llm vs. sast: A technical analysis on detecting coding bugs of gpt4-advanced data analysis," 2025. [Online]. Available: https://arxiv.org/abs/2506.15212

[46] H. Siadati, S. Jafarikhah, E. Sahin, T. Hernandez, E. Tripp, D. Khryashchev, and A. Kharraz, "Devphish: Exploring social engineering in software supply chain attacks on developers," in 2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), 2024, pp. 517–523.

[47] S. Yun, Y. Zhang, Z. Shi, L. Wang, Y. Wang, and Y. Bai, "Large-scale empirical study of secret keys leakage in hugging face spaces," 05 2026.

[48] Z. Li, J. Wu, X. Ling, T. Luo, Z. Rui, and Y. Wu, "The seeds of the future sprout from history: Fuzzing for unveiling vulnerabilities in prospective deep-learning libraries," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 1616–1627.

[49] K. Zhang, S. Wang, J. Han, X. Zhu, X. Li, S. Wang, and S. Wen, "Your fix is my exploit: Enabling comprehensive dl library api fuzzing with large language models," 2025. [Online]. Available: https://arxiv.org/abs/2501.04312

[50] M. Dimjašević, S. Atzeni, I. Ugrina, and Z. Rakamaric, "Evaluation of android malware detection based on system calls," in Proceedings of the 2016 ACM on International Workshop on Security And Privacy Analytics, ser. IWSPA '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1–8. [Online]. Available: https://doi.org/10.1145/2875475.2875487

[51] S. Shakya and M. Dave, "Analysis, detection, and classification of android malware using system calls," 2022. [Online]. Available: https://arxiv.org/abs/2208.06130

[52] J. Zhao, S. Wang, Y. Zhao, X. Hou, K. Wang, P. Gao, Y. Zhang, C. Wei, and H. Wang. (2024) Models are codes:

Towards measuring malicious code poisoning attacks on pre-trained model hubs. https://zenodo.org/records/13850049. Https://zenodo.org/records/13850049.

[53] David Cohen. (2024) Data scientists targeted by malicious hugging face ml models with silent backdoor. https://jfrog.com/blog/data-scientists-targeted-by-malicious-hugging-face-ml-models-with-silent-backdoor/.

[54] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[55] N. Koutroumpouchos, G. Lavdanis, E. Veroni, C. Ntantogian, and C. Xenakis, "Objectmap: detecting insecure object deserialization," in Proceedings of the 23rd Pan-Hellenic Conference on Informatics, ser. PCI '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 67–72. [Online]. Available: https://doi.org/10.1145/3368640.3368680

[56] A. D. Kellas, N. Christou, W. Jiang, P. Li, L. Simon, Y. David, V. P. Kemerlis, J. C. Davis, and J. Yang, "Pickleball: Secure deserialization of pickle-based machine learning models," in Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, ser. CCS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 3341–3355. [Online]. Available: https://doi.org/10.1145/3719027.3765037

[57] Y. Gao, I. Shumailov, and K. Fawaz, "Supply-chain attacks in machine learning frameworks," in Proceedings of Machine Learning and Systems, M. Zaharia, G. Joshi, and Y. Lin, Eds., vol. 7. MLSys, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/file/75bb91b908e6924763c9f2bbe87e921e-Paper-Conference.pdf

[58] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: A system for Large-Scale machine learning," in 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. [Online]. Available: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

[59] H. Ohayon, D. Gilkarov, and R. Dubin, "Safepickle: Robust and generic ml detection of malicious pickle-based ml models," 2026. [Online]. Available: https://arxiv.org/abs/2602.19818

[60] Z. Mousavi, C. Islam, M. A. Babar, A. Abuadbba, and K. Moore, "Detecting misuse of security apis: A systematic review," ACM Comput. Surv., vol. 57, no. 12, Jul. 2025. [Online]. Available: https://doi.org/10.1145/3735968

[61] D. Gibert, C. Mateu, and J. Planes, "The rise of machine learning for detection and classification of malware: Research developments, trends and challenges," Journal of Network and Computer Applications, 03 2020.

[62] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in AI 2016: Advances in Artificial Intelligence, B. H. Kang and Q. Bai, Eds. Cham: Springer International Publishing, 2016, pp. 137–149.

[63] L. Cui, J. Yin, J. Cui, Y. Ji, P. Liu, Z. Hao, and X. Yun, "Api2vec++: Boosting api sequence representation for malware detection and classification," IEEE Transactions on Software Engineering, vol. 50, no. 8, pp. 2142–2162, 2024.

[64] P. Maniriho, A. N. Mahmood, and M. J. M. Chowdhury, "Api-maldetect: Automated malware detection framework for windows based on api calls and deep learning techniques," J. Netw. Comput. Appl., vol. 218, no. C, Sep. 2023. [Online]. Available: https://doi.org/10.1016/j.jnca.2023.103704

[65] X. Chen, Z. Hao, L. Li, L. Cui, Y. Zhu, Z. Ding, and Y. Liu, "Cruparamer: Learning on parameter-augmented api sequences for malware detection," IEEE Transactions on Information Forensics and Security, vol. 17, pp. 788–803, 2022.